

王松宸

2024201594

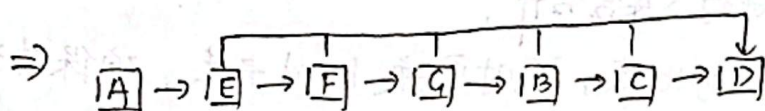
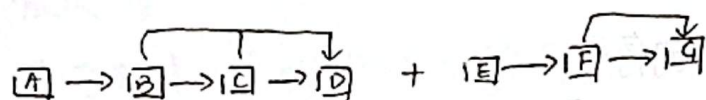
1. 有道理



将链表修改为如图所示的形式，每个集合对象只有 head 指针
链表中每个元素均指向末尾元素，它作为新的集合代表

Find-Set 和 Make-Set 照常为 $O(1)$

对于 weighted-union，每次将较短的链表接在长链表 head 后面，
依然保持摊还代价 $O(\lg n)$



2. a. $\text{extracted} = [4, 3, 2, 6, 8, 1]$

b. I_1, E, I_2, E, I_3, E

$I_{\min}$

$$I_2' = I_1' + I_2$$

因此对于 kj ，和它后面第一个集合合并可得到类似 I_2' 的新集合

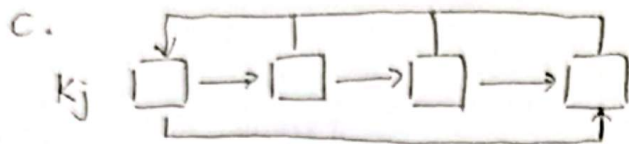
for $i = 1$ to n 从最小的元素遍历，确保是每次 Extract-Min 得到的

如果在 k_{m+1} ，说明它之后无 E ，无法弹出

$\therefore$ 该算法得到的是正确的 extracted



扫描全能王 创建



Off-Line-Minimum (m, n):

for $i = 1$ to n :

$j = \text{Find-Set}(i)$

if $j \neq m+1$:

$\text{extracted}[j] = K_j.\text{head}$

$\text{Union}(j, l)$

Delete K_j

return extracted

其中 Find-Set 返回的是集合的序号

Union 也用序号来进行 2 集合合并，

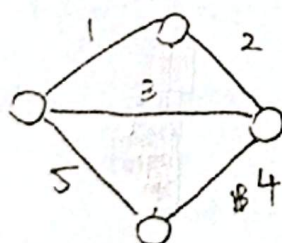
合并过程中进行值的比较，随时更换 head 元素，确保其为集合元素中最小的。

若使用 weighted-union，最坏情况下 ~~单元素 $\log n$ 次指针~~，总共 ~~$O(n \log n)$~~

每个元素修改 $\log n$ 次指针

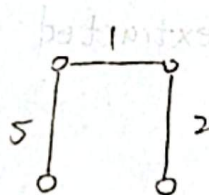
总共 $O(n \log n)$

3. a. 带权无向图，权重各不相同

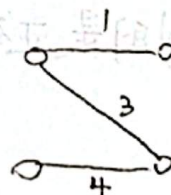


最小生成树]

次优最小生成树



8



8

∴ 次优不唯一



扫描全能王 创建

若有 2 个最小生成树 T_1, T_2

$$\text{令 } x = \min_{(u,v) \in T_1} w(u,v)$$

将 x 加入 T_2 , 则形成一个环

将环中最大权重且依然保持 ST 的边 y 去掉

$\therefore$ 则 T_2' 总权重小于 T_2 且为 ST

$\therefore T_2$ 不是 MST

$\therefore$ MST 只存在一个

b. 将构成 MST 之外的边加入优先队列 Q , 按 w 升序排列

依次将一条边加入 MST, 在形成的环中观察:

若可去掉一条 w 最接近 x 的边并加入 x , 依然形成 ST,

则记录其为新的 ST

最终, w 最小的即为所求 次优最小生成树

思想: 从 $\text{Cut}(S, V-S)$ 中选 w 次小的 crossing edge, 其他均为 lightest

c. 维护一个 $|V| \times |V|$ 的数组 dist 来记录所有的 $\max[u, v]$

Init - $\text{dist}()$

for $i = 1$ to n :

$\text{dist}[i][i] = 0$

for each $(u, v) \in E$:

$\text{dist}[u][v] = w(u, v)$

其余 dist 元素设为 $-\infty$

~~Max~~ DFS-MAX(G, u):

for each v in $u.\text{neighbors}$:

~~dist[u][v]~~

if $v.\text{color} \neq \text{white}$:

$\text{dist}[u][v] = \max(\text{dist}[u][v], w(u, v))$

$v.\text{color} = \text{gray}$

DFS-MAX(G, v)

$v.\text{color} = \text{black}$

V 轮

每轮 $O(V)$

共 $O(V^2)$

Fib heap 实现 Prim 求了

Prim(G, s):

$Q = V \setminus \{s\}$ $A = \emptyset$

$\text{key}(v) = \infty$

while $Q \neq \emptyset$:

$u = \text{Extract-Min}(Q)$

update $u.\text{neighbors}$ 的 key

add u and its edge to A

return A

$n \uparrow$ Extract-Min

$m \uparrow$ Decrease-Key

$O(E + V \lg V)$



d. Fib heap 的 Prim $\Rightarrow T$

~~DFS~~ DFS-MAX(u) 得到 $\text{dist}[1 \dots n][1 \dots n]$

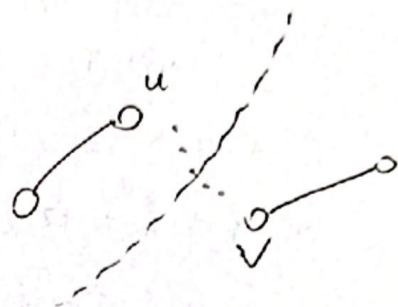
for each $(u, v) \notin T$:

~~$w = \min(w, \text{weight}(u, v) - \max(u, v))$~~

if $\text{weight}(u, v) - \max(u, v) < w$:

$w = \text{weight}(u, v) - \max(u, v)$

$e = (u, v)$



用 e 替换 $\max(u, v)$ 对应的边得到次优最小生成树

只替换1个边: 显然1个的比2个的更小

成环? $\max(u, v)$ 对应的边被切了, 原 T 中 u, v 不连通, (u, v) 加入不成环

4. Dijkstra(G, s):

$Q = G.V$ $S = \emptyset$

~~$s = d = 0$~~ Initialize-Single-Source(G, s)

while $|Q| > 1$:

$u = \text{Extract-Min}(Q)$ $S = S \cup \{u\}$

for $v \in G.\text{adj}[u]$:

relaxation(u, v)

return

是正确的, 前 $|V|-1$ 次从 Q 中弹出 $|V|-1$ 个结点, 求出了 s 到 $|V|-2$ 个结点的最短路径, 第 $|V|-1$ 次更新了 u 的邻居, 使得最后留在 Q 中的点 v 的 $v.d$ 已完全更新好, 所以最后一次遍历不必要

5. Dijkstra(G, s):

$Q = G.V$ $S = \emptyset$

Initialize-Single-Source(G, s)

while buf 不为空:

for $i = 1$ to W :

$j = \text{buf}[i]$ 含结点

for $v \in G.\text{adj}[j]$:

if $v.d > j.d + w(j, v)$:

$\text{new-d} = j.d + w(j, v)$

$\text{buf}[\text{new-d}].\text{add}(v)$

$v.d = \text{new-d}$

遍历视为 Extract-Min $O(W)$ 一轮, 共 V 轮
 $\therefore O(WV)$

新的 relax $\Rightarrow O(E)$ 不变

